MISA®
TIN CẬY - TIỆN ÍCH - TẬN TÌNH

RESTful API

KẾT NỐI - THẤU HIỂU - CHIA SẺ

Nội dung

- Giới thiệu về RESful API
- Các nguyên tắc cơ bản trong RESful API

RESful API là gì?

- RESful API không phải là một loại công nghệ.
- RESful API là một tiêu chuẩn dùng trong việc thiết kế API cho các ứng dụng Web (Web Service).

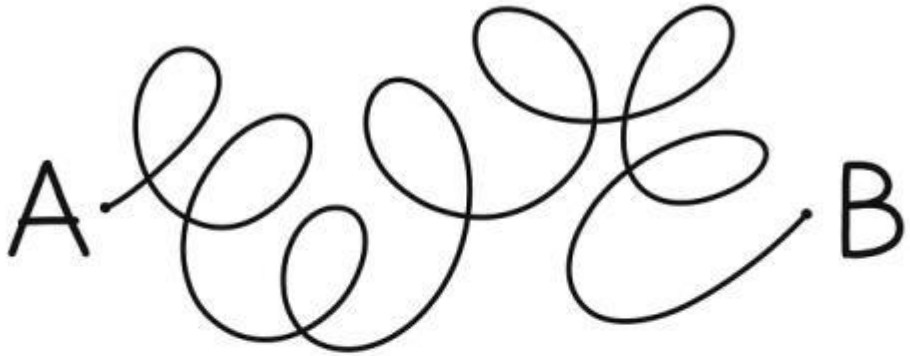
RESful hoạt động như thế nào?

- Hoạt động chủ yếu dựa vào giao thức HTTP.
  - GET: trả về resource
  - POST: Tạo mới một Resource
  - PUT: Cập nhật một Resource
  - DELETE: Xóa một Resource
- RESTful API không sử dụng Session và Cookie. Nó sử dụng một access_token với mỗi request.

Nguyên tắc thiết kế cơ bản trong RESful API

1. Làm mọi thứ thật đơn giản theo nguyên tắc KISS (Keep It Simple Stupid).

/users/checkExist/{email}
/users/register



/users

Nguyên tắc cơ bản

2. Sử dụng danh từ, **không** dùng động từ.

```
/users
```

```
/getAllUsers
```

3. Sử dụng phương thức phù hợp:

- GET - Để lấy tài nguyên, không thay đổi trạng thái tài nguyên.

```
GET http://www.example.com/users
```

- POST - Để tạo tài nguyên.

```
POST http://www.example.com/users
```

Nguyên tắc cơ bản

3. Sử dụng phương thức phù hợp:

- PUT/PATCH - Để cập nhật tài nguyên hoặc một phần tài nguyên:

```
PUT http://www.example.com/users/123 {“name”: “nvan”, “email”: “nvan@gmail.com”}
PATCH http://www.example.com/users/123 {“email”:”vnan@gmail.com”}
```

- DELETE - Để xóa tài nguyên:

```
DELETE http://www.example.com/users/123
```

4. Sử dụng danh từ số nhiều và thống nhất cho toàn bộ dự án.

/users ✅

/user/all ⚠️

Nguyên tắc cơ bản

6. **Sử dụng tham số** (parameter): Giúp cho base URL đơn giản và có thể mở rộng để phục vụ nhiều mục đích khác

```
/users?name='vnan'
```

```
/users/getUsersByName
```

7. **Sử dụng mã HTTP Status Code**:

- 200 OK - Thành công.
- 201 CREATED - Tạo dữ liệu thành công (POST method).
- 400 BAD REQUEST - Input của Client gửi không hợp lệ.
- 401 UNAUTHORIZED - Hệ thống chưa được ủy quyền.

7. Sử dụng mã HTTP Status Code:

- **200** OK - Thành công.
- **201** CREATED - Tạo dữ liệu thành công (POST method).
- **400** BAD REQUEST - Input của Client gửi không hợp lệ.
- **401** UNAUTHORIZED - Hệ thống chưa được ủy quyền.
- **403** FORBIDDEN - Máy chủ từ chối yêu cầu (ngay cả khi Client đã được ủy quyền).
- **404** NOT FOUND - Máy chủ không tìm thấy bất kỳ thứ gì với request-URL này.
- **500** INTERNAL SERVER ERROR - Có lỗi xảy ra phía máy chủ.

8. **Phiên bản (Version)**

- Rất quan trọng, hỗ trợ việc tương thích ngược trong quá trình nâng cấp sau này. Đặt trước tên tài nguyên.
- Áp dụng nguyên tắc KISS và không chứa ký tự đặc biệt.

/**v1**/users ✔

/**v1.1.2**/users ⚠

Nguyên tắc cơ bản

9. **Lọc, sắp xếp, phân trang**:

- Lọc (Filtering): Giảm kết quả truy vấn theo các tham số cụ

```
GET http://www.example.com/users?country=VN
```

-

```
GET http://www.example.com/users?sort=birthdate_date:asc
```

- Phân trang (Paging): Sử dụng khi API trả về dữ liệu lớn. Ví dụ: Sử

```
GET http://www.example.com/users?limit=25&offset=50
```

Nguyên tắc cơ bản

10. Format dữ liệu trả về: JSON

11. Xử lý thông báo lỗi:

- Nội dung trả về cần bao gồm:
  - devMsg - Thông báo cho Dev
  - userMsg - Thông báo cho người dùng cuối (khi tích hợp)
  - errorCode - Mã lỗi nội bộ
  - moreInfo - Hỗ trợ Dev tìm hiểu thêm thông tin về lỗi
  - traceId - Để tra cứu thông tin log (ELK, file log,...)

```
{
  "devMsg": "It was not possible to connect to the redis server(s)",
  "userMsg": "Có lỗi xảy ra! vui lòng liên hệ với MISA.",
  "errorCode": "misa-001",
  "moreInfo": "https://openapi.misa.com.vn/errorcode/misa-001",
  "traceId": "ba9587fd-1a79-4ac5-a0ca-2c9f74dfd3fb"
}
```

DEMO

Thank you!
Any **Questions?**